

# DataKern Data Engineering — Assignment 3

## From Validated Files to Business Data Products

*Continuation of Assignment 1: Olist Pandas Data Exploration, Processing & Validation*

**Important learning rule:** Do not use AI code-generation or coding-agent tools to produce your assignment solution. This includes ChatGPT, Gemini, Claude, GitHub Copilot, Cursor, or similar tools. You may use official documentation and search for errors, but you must write and explain your own solution.

## 1. Your Next Engineering Assignment

You completed your first assignment as a Data Engineer.

Your team lead reviewed your Pandas work and was impressed with how you explored unfamiliar data, documented what you found, processed the files, and validated the results.

But the team has one more problem.

The cleaned files are useful — but they are still files.

Finance wants consistent sales numbers. Operations wants to investigate delivery performance. Other engineers may need to reuse the same customer, order, product, and review information.

If every person opens CSV files and performs their own joins and calculations, the team will eventually get different answers to the same question.

Your team lead comes back to you:

**"The data is now clean enough to work with. Your next job is to turn it into a small, trusted data system. Keep the source evidence safe, organise the business entities properly, create intermediate working steps where they help, and finally produce data that a business team can actually use."**

That is your assignment.

You are not starting a new project.

You are continuing the engineering work you already did.

### The journey so far

```
Assignment 1
Olist CSV files
  ↓
Explore
  ↓
Process with Pandas
  ↓
Validate
  ↓
Processed / trusted files
```

### Your next stage

```
Validated files
  ↓
RAW
  ↓
```

CLEAN  
↓  
STAGING  
↓  
CURATED  
↓  
Business answers

Your goal is **not** to move CSV files into PostgreSQL as quickly as possible.

Your goal is to build a small data system where:

- the source evidence is preserved;
- reusable business entities are clearly modelled;
- transformations are broken into understandable steps;
- intermediate work is separated from final business products;
- every important stage is validated;
- the final answers have a clearly defined meaning;
- another engineer can understand what you built and why.

## 2. What You Are Building

You will work with the same Olist dataset from Assignment 1.

You will use:

- **Python**
- **Pandas**
- **PostgreSQL**
- **SQL**
- **Jupyter Notebook**
- **Git/GitHub**

For this assignment, use **one main notebook** as your working environment.

You do not need to create a separate Python loading script yet.

The purpose is to strengthen your Python + SQL thinking before introducing more deployment and automation complexity.

You will alternate between Python/Pandas and SQL depending on which tool is appropriate for the task.

### The principle

***Do not ask "Should I use Python or SQL?" first. Ask "Where does this piece of work make the most sense?"***

For example:

| Activity                                | A natural choice | Why                                                   |
|-----------------------------------------|------------------|-------------------------------------------------------|
| Read an existing CSV                    | Pandas           | You already know this from Assignment 1               |
| Inspect columns and values              | Pandas           | Fast exploration                                      |
| Prepare a dataframe for loading         | Pandas           | Familiar processing workflow                          |
| Create PostgreSQL schemas               | SQL              | The database owns its structure                       |
| Create table contracts                  | SQL              | Types and constraints belong in the database          |
| Join relational tables                  | SQL              | The database is designed for relational operations    |
| Build intermediate staging tables       | SQL              | These are database transformation steps               |
| Aggregate business metrics              | SQL              | The transformation belongs close to the governed data |
| Check row counts                        | Python or SQL    | Either can provide useful evidence                    |
| Investigate unexpected database results | SQL + Python     | Use whichever helps you understand the problem        |

The objective is not to prove that Python is better than SQL, or SQL is better than Python.

The objective is to learn to choose deliberately.

## 3. The Four-Layer Mental Model

Before writing any code, understand what each schema is responsible for.

You will create four PostgreSQL schemas:

```
raw
clean
staging
curated
```

Think about them using four questions.

| Schema  | Question                                                   | Purpose                                       |
|---------|------------------------------------------------------------|-----------------------------------------------|
| raw     | What did we receive?                                       | Preserve source evidence                      |
| clean   | What trustworthy business entities do we have?             | Store reusable, cleaned relational data       |
| staging | What intermediate work is needed to create a data product? | Hold temporary/working transformation results |
| curated | What final business data product are we providing?         | Serve defined business questions              |

This distinction is one of the most important things you should learn in this assignment.

### 3.1 RAW

RAW is close to the source.

Your original CSV files remain the source evidence.

The database representation in `raw` should preserve what you received.

Do not perform business cleaning simply because you are uncomfortable with a source value.

If the source contains:

- missing values;
- unusual text;
- unexpected categories;
- values that need interpretation;

RAW should not quietly rewrite them.

Think:

*"What did we receive?"*

### 3.2 CLEAN

CLEAN is different.

Here you want trustworthy, reusable business entities.

For the Olist data, this may include entities such as:

```
clean.customers
clean.orders
clean.order_items
clean.products
clean.reviews
```

These tables should have meaningful data types, validated keys, and understandable relationships.

CLEAN should **not** be designed around one specific Finance question.

It should remain useful when another team asks a different question tomorrow.

Think:

***"What trusted business entities do we have?"***

## 3.3 STAGING

This is where many beginners get confused.

A staging table is an **intermediate working table**.

It exists because a transformation becomes easier to understand, validate, debug, or reuse when you break it into steps.

Staging is not the final answer.

It is not your cleaned relational model.

It is not a business-facing product.

Think:

***"What work do I need to do between trusted entities and the final answer?"***

You will create at least **two meaningful staging tables for each business problem**.

That requirement is deliberate.

We do not want a giant SQL query that joins six tables, calculates everything, filters everything, and produces the final answer in one shot.

We want you to learn how to break transformation logic into understandable engineering steps.

## 3.4 CURATED

CURATED contains the final data products.

A curated table should exist because a business requirement exists.

It should have:

- a clearly stated purpose;
- a clearly stated grain;
- defined metrics;
- understandable columns;
- validated results.

Think:

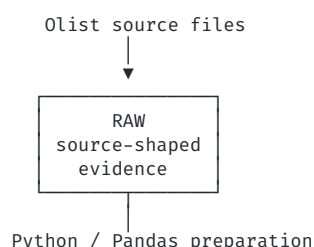
***"What final data product are we providing?"***

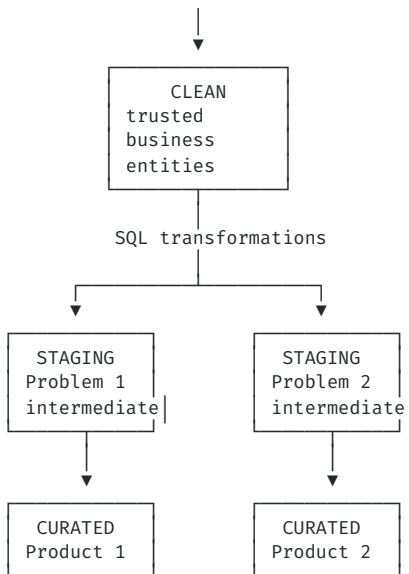
Do not put intermediate calculations in **curated** simply because you created them.

If a table is only useful while building another table, it probably belongs in **staging**.

## 4. The Architecture You Should Aim For

Your final architecture should look conceptually like this:





Notice something important:

**RAW does not feed back from CLEAN.**

**CLEAN does not become a dumping ground for staging tables.**

**STAGING does not become CURATED simply because it contains useful-looking data.**

Each layer has a responsibility.

## 5. The Two Business Problems

Your team has two initial requirements.

You will create **two final curated data products**.

You will not create ten dashboards or twenty reports.

The purpose is to learn how a Data Engineer moves from business requirements to data products.

### Business Problem 1 — Finance

Finance wants to understand sales performance over time and across product categories.

The business question is:

***"How is item sales value changing across product categories over time?"***

You need to define exactly what this means.

Do not immediately write:

```
SUM(price)
```

First ask:

- What does one source row represent?
- Which table contains the sales event?
- What is the relationship between an order and its items?
- Which timestamp should determine the reporting month?
- What does "sales value" include?
- Should freight be included?
- Which order statuses should be included?
- What should happen when a product category is missing?
- What does one final output row represent?

The exact metric definition is part of your engineering work.

**Your final Finance data product should support analysis such as:**

```
Month
Product Category
Item Sales Value
Distinct Orders
```

You may add other useful measures if they are justified and validated.

But do not add columns simply because you can.

---

## Business Problem 2 — Operations

Operations wants to understand delivery performance.

The business question is:

***"How is delivery performance changing, and where are late deliveries occurring?"***

Again, do not jump directly into SQL.

Ask:

- Which table contains the promised delivery date?
- Which table contains the actual delivery date?
- What does one order represent?
- Can an order have multiple item rows?
- Which date should be used for the reporting period?
- What exactly counts as "late"?
- What should happen when a delivery date is missing?
- Should cancelled orders be treated as late?
- Is the final measure based on orders or order items?
- Which geography or customer attribute is useful for the operational question?

You must make the definition visible.

**Your final Operations data product might contain concepts such as:**

```
Reporting Period
Location / Region
Total Orders
Delivered Orders
Late Orders
Late Delivery Rate
```

These are **directional hints**, not a finished schema.

You must inspect the actual Olist data and decide which fields support the requirement.

---

## 6. Your Engineering Thought Process

This assignment is not about memorising SQL.

For every transformation, practise the following thought process.

---

### Step 1 — Understand

Ask:

*What exactly is the business asking?*

Write the requirement in plain English.

---

## Step 2 — Identify the grain

Ask:

*What should one row represent at this stage?*

Write it down before creating the table.

For example:

One row represents one \_\_\_\_.

If you cannot complete that sentence confidently, stop.

You are not ready to write the transformation.

---

## Step 3 — Identify the source entities

Ask:

*Which trusted CLEAN tables contain the information I need?*

Do not start joining everything.

Identify only the entities that contribute to the requirement.

---

## Step 4 — Understand the relationships

Ask:

*How are these entities connected?*

Use the keys you validated earlier.

Do not join columns simply because their names look similar.

---

## Step 5 — Predict the row count

Before executing the query, make a rough prediction.

For example:

*"If I start from one row per order and join to order items, I expect the number of rows to increase because one order may have multiple items."*

This habit is extremely important.

A good engineer predicts behaviour before looking at the result.

---

## Step 6 — Create one meaningful intermediate result

Ask:

*What is the first useful piece of information I can create?*

This becomes a staging table.

Do not create a staging table just to satisfy the assignment.

Create it because it gives the transformation a meaningful step.

---

## Step 7 — Validate

Ask:

*Does this staging table contain what I expected?*

Check:

- row count;
- nulls;
- duplicate grain;
- sample records;
- totals;
- relationship behaviour.

---

## Step 8 — Continue the transformation

Now create the next staging table.

Again:

What does one row mean?  
Why do I need this table?  
What changed from the previous stage?  
How can I validate it?

---

## Step 9 — Create the final curated product

Only after the staging steps are understood should you create the curated table.

The final table should be easy for a business user to understand.

---

## Step 10 — Reconcile

Your final question is:

*Can I prove that the curated result agrees with the underlying CLEAN data under the same business rules?*

If you cannot reconcile it, the work is not finished.

---

## 6.1 Core Validation Strategy

Validation is not a final checkbox. Validate after every important stage.

At minimum check:

- **Row count:** How many rows? Was that expected?
- **Grain:** Is the declared grain actually unique?
- **Nulls:** Are important keys or measures missing?
- **Relationships:** Did a join create unexpected unmatched records?

- **Sample records:** Inspect a few rows manually.
- **Totals:** Compare totals against the previous stage.

## 7. Project Setup

Continue using your Assignment 1 repository.

Do not start a separate unrelated project.

A suggested structure is:

```
olist-data-engineering/  
├── data/  
│   ├── raw/  
│   └── processed/  
├── notebooks/  
│   └── assignment_3_warehouse.ipynb  
├── docs/  
│   ├── data_model.md  
│   ├── business_requirements.md  
│   └── validation_evidence.md  
├── sql/  
│   └── optional_sql_notes.sql  
├── README.md  
├── requirements.txt  
└── .gitignore
```

You can set up this structure with the following commands:

```
mkdir -p olist-data-engineering/data/{raw,processed}  
mkdir -p olist-data-engineering/notebooks  
mkdir -p olist-data-engineering/docs  
mkdir -p olist-data-engineering/sql  
touch olist-data-engineering/notebooks/assignment_3_warehouse.ipynb  
touch olist-data-engineering/docs/data_model.md  
touch olist-data-engineering/docs/business_requirements.md  
touch olist-data-engineering/docs/validation_evidence.md  
touch olist-data-engineering/sql/optional_sql_notes.sql  
touch olist-data-engineering/README.md  
touch olist-data-engineering/requirements.txt  
touch olist-data-engineering/.gitignore
```

You may organise your files differently if you have a clear reason.

The important thing is that another engineer can understand the project.

### 7.1 PostgreSQL setup

Create a PostgreSQL database for this assignment.

For example:

```
CREATE DATABASE olist_warehouse;
```

Inside it, create the required schemas:

```
CREATE SCHEMA raw;  
CREATE SCHEMA clean;  
CREATE SCHEMA staging;  
CREATE SCHEMA curated;
```

Do not create separate PostgreSQL databases for each layer.  
The layers are schemas inside the same database.

## 7.2 Python environment

Use the virtual environment from your previous assignment if appropriate.

Install the packages needed for this work.

For example:

```
python -m pip install pandas psycpg[binary] python-dotenv sqlalchemy jupyter
```

You may use another PostgreSQL-compatible Python library if your instructor has already introduced it.

The important thing is that Python can:

1. read your files;
2. connect to PostgreSQL;
3. load data;
4. execute SQL;
5. retrieve validation results.

## 7.3 Protect credentials

Do not place passwords directly in the notebook.

Use environment variables or a `.env` file.

Example:

```
DATABASE_URL=postgresql://<username>:<password>@localhost:<port>/olist_warehouse
```

Add `.env` to `.gitignore`.

Create `.env.example` without real credentials.

### Pause and Observe

Why is it dangerous to put a real password inside:

```
connection = "postgresql://user:realpassword@... "
```

and commit that notebook to GitHub?

Write your answer in the notebook.

## 8. Notebook Structure and Flow

Use one notebook as your main working environment. Your final notebook should read approximately like an engineering investigation.

```
01. BUSINESS REQUIREMENTS
```

```
What are Finance and Operations asking? What assumptions exist?
```

```
02. INSPECT EXISTING DATA
```

```
Load Assignment 1 outputs. Confirm files and columns.
```

### 03. CONNECT TO POSTGRESQL & CREATE SCHEMAS

Connect safely. Do not expose credentials. Create raw, clean, staging, curated.

### 04. LOAD RAW & BUILD CLEAN

Read → load → validate RAW.

Design → create → prepare → load → validate CLEAN.

### 05. FINANCE PIPELINE

Requirement → Grain → CLEAN sources → STAGING 1 → STAGING 2 → CURATED → Business query (Validate at every step)

### 06. OPERATIONS PIPELINE

Requirement → Grain → CLEAN sources → STAGING 1 → STAGING 2 → CURATED → Business query (Validate at every step)

### 07. FINAL VALIDATION & REFLECTION

Reconcile curated results. What did you learn? What would you improve?

Use Markdown cells to explain your thinking. Use Python cells for Python/Pandas work. Use SQL cells for SQL work. Your notebook should tell the story of the pipeline.

## 9. Stage 1 — Load RAW

### Goal

Get the original source data into PostgreSQL while preserving source evidence.

Create the schemas first:

```
raw
clean
staging
curated
```

Then create the RAW tables.

You are working with the Olist source files from Assignment 1.

At minimum, investigate these:

```
olist_orders_dataset.csv
olist_customers_dataset.csv
olist_order_items_dataset.csv
olist_products_dataset.csv
olist_order_reviews_dataset.csv
```

### RAW design rules

RAW should be:

- close to the source;
- source-aligned;
- traceable;
- reloadable;
- minimally interpreted.

Do not use RAW to fix business problems.

For example, if a source category is missing:

Do not silently replace it with "Unknown"

unless you have explicitly decided that this is part of a later transformation.  
RAW should answer:

*"What did the source give us?"*

## Loading with Python

Use Python/Pandas in the notebook to:

1. identify the input file;
2. read it;
3. inspect the dataframe;
4. connect to PostgreSQL;
5. load the appropriate RAW table;
6. check the database row count.

You may research the PostgreSQL/Python loading methods.

Look for concepts such as:

```
pandas DataFrame → PostgreSQL table  
bulk loading  
COPY  
transactions
```

Do not blindly copy code from a tutorial.

Understand what the operation does.

## RAW validation

For each RAW table, compare:

```
source file row count  
vs  
database row count
```

If they differ, stop.

Investigate before continuing.

### Pause and Observe: Questions to ask

- Did I count the header?
- Did I accidentally load the file twice?
- Did parsing fail?
- Did the database transaction commit?
- Did I load the correct file into the correct table?

## 10. Stage 2 — Build CLEAN

Now the work becomes more interesting.

Your previous assignment produced processed data.

You now need to represent that trusted data as relational business entities.

Think:

```
Assignment 1 processed files
↓
CLEAN relational model
```

Possible entities include:

```
clean.customers
clean.orders
clean.order_items
clean.products
clean.reviews
```

Use your Assignment 1 observations to confirm the actual structure.

## 10.1 Design before loading

For every CLEAN table, document:

```
Table:
Purpose:
Grain:
Primary key:
Foreign keys:
Important columns:
Important data types:
Important constraints:
```

Do not create constraints simply because they look professional.

Ask:

***What real-world rule does this constraint represent?***

## 10.2 Load dependency-aware

Think about the relationships.

For example:

```
customer
↓
order
↓
order item
↓
product
```

This is only a mental example.

Your actual load order must come from the relationships you designed.

If a child table references a parent table, ask:

***"Can I load the child before the parent and still satisfy the relationship?"***

Usually, the answer tells you something about dependency order.

## 10.3 CLEAN validation

Validate:

## Identity

```
Are declared primary keys non-null?  
Are they unique?
```

## Relationships

```
Do foreign-key values find their parents?
```

## Types

```
Are dates actually dates?  
Are prices numeric?
```

## Grain

```
Does the table contain exactly one row at the intended grain?
```

## Reconciliation

```
Does the CLEAN row count agree with the validated processed source?
```

Record the results.

# 11. Business Problem 1 — Finance

Now the real engineering work begins.

Finance wants:

*How is item sales value changing across product categories over time?*

Your job is to turn that sentence into a data product.

Do not start with the final SQL.

## 11.1 First write the requirement

Create:

```
touch docs/business_requirements.md
```

Write the following content inside it:

```
Business problem:  
Business user:  
Business question:  
Metric:  
Time definition:  
Included records:  
Excluded records:  
Unknown/missing-value treatment:
```

Expected final grain:  
Assumptions:

If you cannot fill one of these fields, investigate the data first.

## 12. Finance — Think About the Grain

This is your first major design decision.

Your final product might have a grain like:

One row = one category for one month

But do not copy that statement without understanding it.

Ask:

*If January has 200 order items from three categories, how many rows should the final table contain?*

The answer depends on the final grain.

Now ask:

*What happens before the data reaches that final grain?*

This is where staging becomes useful.

## 13. Finance — Staging Table 1 Hint

You need at least two meaningful staging tables.

### Hint — first staging table

Start from the most detailed business event that contains the measure you need.

Ask:

*"What is the smallest meaningful business record from which I can calculate sales?"*

Then identify:

- the table containing the item-level value;
- the table containing product/category information;
- the relationship connecting them;
- the timestamp you need;
- the order identifier.

Your first staging table should make the detailed sales records easier to reason about.

### Do not ask:

*"What SQL query should I write?"*

Ask:

*"What should one row of my first staging table represent?"*

Write that sentence before writing SQL.

For example:

One row in staging table 1 represents one \_\_\_\_\_.

Then decide which columns are needed.

### Validation hint

After creating the first staging table, check:

How many rows did I expect?  
How many rows did I get?  
Did a JOIN increase the row count?  
Was that increase expected?  
Are important keys null?  
Are categories missing?

If a join unexpectedly multiplies rows, stop and investigate.

## 14. Finance — Staging Table 2 Hint

Now ask:

*"What work remains before I can produce the business answer?"*

Your second staging table should move the data closer to the final business grain.

Possible transformation ideas include:

- deriving a reporting month;
- applying documented business filters;
- resolving a category representation;
- calculating a row-level measure;
- reducing the detailed event data into a useful intermediate grain.

Do not combine every operation into one huge transformation.

The staging table should have a reason to exist.

Write:

Why does this staging table exist?  
What does one row represent?  
What changed from staging table 1?  
How will I validate it?

### Hint

If your final product is:

category + month

then your second staging table should probably be much closer to that grain than the first one.

But you must decide exactly where aggregation belongs.

## 15. Finance — Curated Table Hint

Now design the final curated table.

The table should answer Finance's recurring question without requiring Finance to understand your entire relational model.

Think:

```
curated.finance_ ...
```

or another meaningful name.

Your final table should have a clear grain.

Write:

```
One row represents one _____.
```

Then define the measures.

For example, you might need:

```
reporting_month  
category  
sales_value  
distinct_orders
```

These are hints about the type of information the business needs, not a finished answer.

## Final query hint — Finance

The final query should be conceptually simple:

```
READ from curated  
  ↓  
filter or order if necessary  
  ↓  
present the business result
```

If your Finance query still requires five joins and a complicated transformation, ask yourself:

***"Did I fail to build the curated product properly?"***

The purpose of CURATED is to make the recurring business question easier.

## 16. Finance — Final Query Thinking Exercise

Before writing the final SQL, answer these questions in Markdown:

1. What is the grain of my curated table?
2. What does `sales_value` mean?
3. Which timestamp determines the month?
4. Which records are excluded?
5. Can one order contribute multiple rows to my detailed staging table?
6. If so, how will I avoid counting the order multiple times?
7. What happens to products without a category?
8. How will I prove that the curated total matches the relevant CLEAN data?

Only after answering these should you write the query.

## 17. Business Problem 2 — Operations

Operations asks:

*How is delivery performance changing, and where are late deliveries occurring?*

This problem is intentionally different from Finance.

Finance teaches you to think about:

events → measures → aggregation

Operations should make you think about:

orders → dates → delivery status → business classification → aggregation

Do not assume the same staging design will work.

## 18. Operations — Start With the Business Definition

Define:

What is an order?  
What is delivered?  
What is late?  
Which date defines the reporting period?  
What happens when the delivery date is missing?  
Which order statuses should be included?  
What location attribute will be used?  
What is the final reporting grain?

A simple definition such as:

*"Late means actual delivery happened after the estimated delivery date."*

may be reasonable, but you must inspect the data and document whether it applies safely to the records you use.

## 19. Operations — Staging Table 1 Hint

Start at the order grain if the business metric is fundamentally about orders.

Ask:

*"What information do I need to decide whether an order was delivered late?"*

Identify the required fields.

Think about:

order identity  
order status  
purchase date  
estimated delivery date  
actual delivery date  
customer/location information

Your first staging table should gather the evidence required to classify delivery performance.

## Pause and Observe

Do not calculate a "late delivery rate" yet.

First build a table where you can inspect individual orders and understand the classification.

This is a good example of why staging exists.

---

## 20. Operations — Staging Table 2 Hint

Now ask:

*"What intermediate result would make the final business aggregation straightforward?"*

Your second staging table might contain a derived classification such as:

```
late
on_time
not_delivered
unknown
```

That is a hint about the type of transformation, not the required exact design.

You must decide whether such categories are appropriate.

For every derived field, ask:

*"What evidence supports this classification?"*

And:

*"What happens at the boundary?"*

For example:

```
actual_date = estimated_date
actual_date < estimated_date
actual_date > estimated_date
actual_date IS NULL
```

These are not just SQL conditions.

They are business rules.

---

## 21. Operations — Curated Product Hint

Your final Operations product should make a recurring operational question easy to answer.

Potential dimensions include:

```
month
customer state
customer city
delivery classification
```

Potential measures include:

```
total orders
delivered orders
```

```
late orders
late delivery rate
```

Do not implement all of them automatically.

Choose the dimensions and measures that are justified by the business question.

Your final curated table should be simple enough that an analyst can query it without rebuilding the underlying business logic.

## 22. Operations — Final Query Hint

The final query should read like a business question.

For example, conceptually:

```
Which periods or locations have a high number of late deliveries?
```

Your final SQL should not need to rediscover:

- what "late" means;
- which dates to compare;
- how orders connect to customers;
- which statuses are valid.

Those decisions should already have been handled in your pipeline.

This is one of the biggest differences between a transformation pipeline and a random SQL query.

## 23. What You Must Not Do

This assignment is deliberately scoped.

Do **not** add:

- Airflow;
- Azure Data Factory;
- CI/CD;
- Docker;
- Kubernetes;
- cloud deployment;
- dashboards;
- APIs;
- streaming;
- new datasets.

Those are valuable topics, but they are not the problem we are solving today.

Manual notebook execution is completely acceptable.

The pipeline can be:

```
Open notebook
↓
Run sections in order
↓
Validate
↓
Review results
```

The engineering quality comes from your design and validation — not from adding more infrastructure.

## 24. Deliverables & Submission Checklist

By the end of the assignment, your repository should contain the following deliverables, and you should be able to answer the following checklist questions.

## Deliverables

1. **Business Requirements Document:** Finance/Operations requirements, metric definitions, assumptions, grains.
2. **Data Model Document:** Purpose of RAW, CLEAN, STAGING, CURATED. ERD, relationships, constraints.
3. **PostgreSQL Database:** `olist_warehouse` containing the 4 schemas.
4. **Notebook/Pipeline:** Showing RAW loading, CLEAN building, Finance/Operations pipelines (at least 2 staging tables each), final CURATED products, and validation evidence.

## Final Submission Checklist

- **Understanding:** I can explain the requirements, assumptions, and grain of every important table.
- **RAW/CLEAN:** RAW preserves source evidence. CLEAN contains reusable business entities with defined primary/foreign keys and data types.
- **STAGING:** I have at least two meaningful staging tables for each problem, each with a documented grain and purpose. Staging is not a dumping ground.
- **CURATED:** I created one final product per problem that answers a specific business need.
- **SQL & Validation:** I understand every JOIN and GROUP BY. I validated row counts, keys, nulls, and relationships at every stage. I reconciled CURATED with underlying data.

## 25. Git Checkpoints

Do not wait until the end to commit everything.

Use small meaningful commits.

For example:

```
git add .
git commit -m "Define warehouse business requirements"
git push
```

Then:

```
git add .
git commit -m "Create PostgreSQL warehouse schemas"
git push
```

Then:

```
git add .
git commit -m "Load and validate raw data"
git push
```

Then:

```
git add .
git commit -m "Build clean relational model"
git push
```

Then:

```
git add .
git commit -m "Build finance data product"
git push
```

Then:

```
git add .
git commit -m "Build operations data product"
git push
```

Then:

```
git add .
git commit -m "Add validation evidence and analysis"
git push
```

Your commit history should tell the story of how the solution evolved.

## 26. A Final Challenge From Your Team Lead

Before you submit, imagine that your team lead opens your notebook six months from now. They ask you:

- "Why did you build it this way?"
- "What does one row in this table represent?"
- "Why did you create this staging table?"
- "How do you know this number is correct?"

If you can trace your logic backwards from the final Business Question to the Raw Source Evidence, you have succeeded. That traceability is the real outcome of this assignment.

## 27. DataKern Engineering Principles

Keep these principles visible while you work.

| Principle                     | What it means in this assignment                                  |
|-------------------------------|-------------------------------------------------------------------|
| Understand before building    | Translate the business requirement before writing SQL.            |
| Problem before technology     | PostgreSQL is a tool; the business requirement comes first.       |
| Grain before query            | Know what one row represents before joining or aggregating.       |
| Raw data is evidence          | Preserve the source rather than silently rewriting it.            |
| Clean data should be reusable | CLEAN represents trusted business entities, not one report.       |
| Staging should have a reason  | Break complex transformations into meaningful, explainable steps. |
| Curated means purpose-built   | A curated product exists for a defined business need.             |
| Validate continuously         | Do not wait until the final query to discover a broken join.      |
| Make assumptions visible      | An explicit assumption is safer than hidden logic.                |
| Predict before you run        | Think about row counts and grain before inspecting results.       |
| Explain why, not just what    | Every important design decision should have a reason.             |
| Build incrementally           | Small stages are easier to debug and trust.                       |
| Keep work understandable      | Another engineer should be able to follow your notebook.          |

## 28. Final Thought

Your first assignment taught you how to take unfamiliar data and make it trustworthy.

This assignment asks you to go one step further.

You are no longer just working with files.

You are beginning to design a **data system**.

The important shift is this:

Before:

"I have a CSV and need to process it."

Now:

"I have a business requirement.  
I need to understand the data.  
I need to design the model.  
I need to decide where each transformation belongs.  
I need to validate every important step.  
I need to produce a data product whose meaning I can explain."

That is the mindset we want you to develop.

*Don't focus on getting the answer. Focus on learning how to engineer the solution. — Uma Kiran | DataKern | [www.data-kern.com](http://www.data-kern.com)*